

# Machine Learning Fundamentals

## Week 4 – Task 1

### Building a Machine Learning Pipeline with Feature Engineering

**Prepared By:** Abdul Raheem

**Course:** Machine Learning Fundamentals

**Organization:** Neurofive Solutions

**Dataset:** Titanic – Machine Learning from Disaster

---

#### Problem Type

Binary Classification

This project demonstrates how to build a professional Machine Learning pipeline using Scikit-learn. The pipeline combines preprocessing, feature engineering, and model training into one reusable workflow, reducing the risk of data leakage and ensuring consistent preprocessing during both training and prediction.

#### Objectives

The objectives of this notebook are:

- Load the Titanic dataset.
- Perform basic preprocessing.
- Engineer new features.
- Separate numerical and categorical features.
- Build a preprocessing pipeline using ColumnTransformer.
- Apply StandardScaler to numerical features.
- Apply OneHotEncoder to categorical features.
- Train a Logistic Regression model using a Pipeline.
- Evaluate the pipeline.

- Save the trained pipeline using Joblib.

```
import pandas as pd
import numpy as np

import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder
from sklearn.preprocessing import StandardScaler
from sklearn.impute import SimpleImputer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score
from sklearn.metrics import classification_report

import joblib
```

## Loading the Dataset

The Titanic dataset is loaded using Pandas.

```
df = pd.read_csv(r"D:\Machine Learning Fundamentals
Course\Titanic_Dataset\train.csv")

df.head()
```

|   | PassengerId | Survived | Pclass | Name                                              | Sex    | Age  | SibSp | Parch |
|---|-------------|----------|--------|---------------------------------------------------|--------|------|-------|-------|
| 0 | 1           | 0        | 3      | Braund, Mr. Owen Harris                           | male   | 22.0 | 1     | 0     |
| 1 | 2           | 1        | 1      | Cumings, Mrs. John Bradley (Florence Briggs Th... | female | 38.0 | 1     | 0     |
| 2 | 3           | 1        | 3      | Heikkinen, Miss. Laina                            | female | 26.0 | 0     | 0     |
| 3 | 4           | 1        | 1      | Futrelle, Mrs. Jacques Heath (Lily May Peel)      | female | 35.0 | 1     | 0     |

|   | PassengerId | Survived | Pclass | Name                           | Sex  | Age  | SibSp | Parch |
|---|-------------|----------|--------|--------------------------------|------|------|-------|-------|
| 4 | 5           | 0        | 3      | Allen, Mr.<br>William<br>Henry | male | 35.0 | 0     | 0     |

```
df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 12 columns):
#   Column          Non-Null Count  Dtype
---  -
0   PassengerId      891 non-null    int64
1   Survived         891 non-null    int64
2   Pclass           891 non-null    int64
3   Name             891 non-null    str
4   Sex              891 non-null    str
5   Age              714 non-null    float64
6   SibSp            891 non-null    int64
7   Parch           891 non-null    int64
8   Ticket           891 non-null    str
9   Fare             891 non-null    float64
10  Cabin            204 non-null    str
11  Embarked         889 non-null    str
dtypes: float64(2), int64(5), str(5)
memory usage: 83.7 KB
```

## Feature Engineering

Two additional features are created to improve the predictive capability of the model.

```
df["FamilySize"] = df["SibSp"] + df["Parch"] + 1
df["IsAlone"] = (df["FamilySize"] == 1).astype(int)
```

## Selecting Features

The target variable is **Survived**.

Selected numerical and categorical features are used to build the machine learning pipeline.

```
features = [
    "Pclass",
    "Sex",
    "Age",
    "Fare",
    "Embarked",
```

```

        "FamilySize",
        "IsAlone"
    ]

X = df[features]

y = df["Survived"]

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42
)

```

## Identifying Numerical and Categorical Features

The preprocessing pipeline applies different transformations to numerical and categorical features.

```

numeric_features = [
    "Age",
    "Fare",
    "FamilySize",
    "IsAlone"
]

categorical_features = [
    "Pclass",
    "Sex",
    "Embarked"
]

```

## Building the Preprocessing Pipeline

A ColumnTransformer is used to apply:

- StandardScaler to numerical columns.
- OneHotEncoder to categorical columns.
- Missing value imputation where required.

```

numeric_transformer = Pipeline(
    steps=[
        ("imputer", SimpleImputer(strategy="median")),
        ("scaler", StandardScaler())
    ]
)

```

```

    ]
)

categorical_transformer = Pipeline(
    steps=[
        ("imputer", SimpleImputer(strategy="most_frequent")),
        ("encoder", OneHotEncoder(handle_unknown="ignore"))
    ]
)

preprocessor = ColumnTransformer(
    transformers=[
        ("num", numeric_transformer, numeric_features),
        ("cat", categorical_transformer, categorical_features)
    ]
)

```

## Building the Machine Learning Pipeline

The preprocessing pipeline and Logistic Regression model are combined into a single reusable Pipeline object.

```

pipeline = Pipeline(
    steps=[
        ("preprocessor", preprocessor),
        ("classifier", LogisticRegression(max_iter=1000))
    ]
)

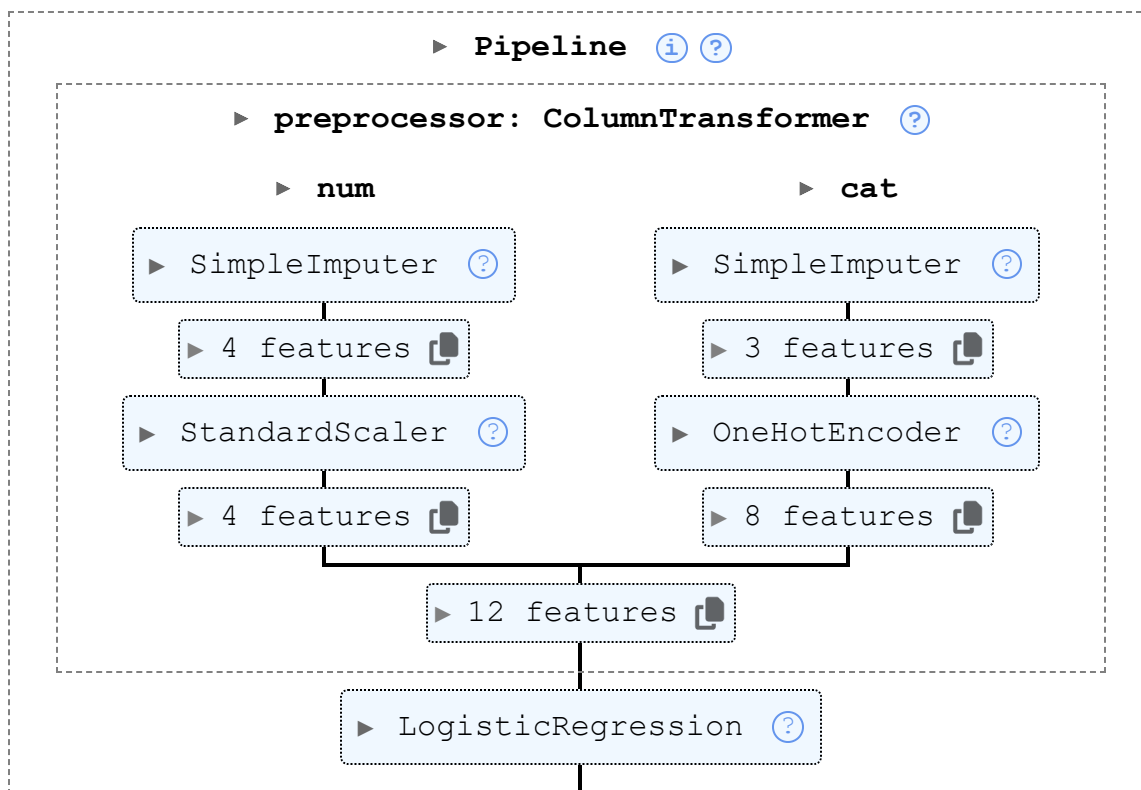
```

## Training the Pipeline

```

pipeline.fit(X_train, y_train)

```



## Model Prediction

```
y_pred = pipeline.predict(X_test)
```

## Model Evaluation

```
accuracy = accuracy_score(y_test, y_pred)
```

```
print("Accuracy:", accuracy)
```

```
Accuracy: 0.7988826815642458
```

```
print(classification_report(y_test, y_pred))
```

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| 0            | 0.81      | 0.87   | 0.83     | 105     |
| 1            | 0.79      | 0.70   | 0.74     | 74      |
| accuracy     |           |        | 0.80     | 179     |
| macro avg    | 0.80      | 0.78   | 0.79     | 179     |
| weighted avg | 0.80      | 0.80   | 0.80     | 179     |

# Pipeline Performance

The pipeline combines preprocessing and model training into a single workflow.

Compared with the manual preprocessing approach used in previous tasks, the pipeline produces consistent preprocessing while minimizing the risk of data leakage and simplifying deployment.

## Saving the Pipeline

The trained pipeline is saved using Joblib for future predictions without retraining.

```
joblib.dump(pipeline, "titanic_pipeline.pkl")

['titanic_pipeline.pkl']

loaded_pipeline = joblib.load("titanic_pipeline.pkl")

loaded_pipeline.predict(X_test.head())

array([0, 0, 0, 1, 1])
```

## Feature Engineering Results

Two engineered features were introduced:

- **FamilySize:** Total number of family members traveling together.
- **IsAlone:** Indicates whether a passenger was traveling alone.

These additional features provide more meaningful information to the model and can improve prediction performance compared to using the original dataset alone.

## Conclusion

In this project, a complete Machine Learning pipeline was developed using Scikit-learn.

The pipeline integrates preprocessing, feature engineering, and Logistic Regression into a single reusable workflow. Numerical features were scaled, categorical features were encoded, and engineered features were added to improve predictive performance.

Finally, the trained pipeline was saved using Joblib, making it ready for deployment and future predictions without repeating preprocessing or retraining steps.